

Regular Expressions and Regular Grammars

Algebraic Descriptions and Grammatical Foundations

Theory of Computation Slide Sets

Table of Contents

- 1 Regular Expressions (RE)
- 2 RE to NFA (Thompson Construction)
- 3 FA to RE (State Elimination)
- 4 Regular Grammars
- 5 The Big Picture & Applications
- 6 Review and Exercises

By the end of this slide set, you should be able to:

- interpret a regular expression as a language and apply precedence rules correctly;
- convert a regular expression to an equivalent ϵ -NFA;
- convert a finite automaton to a regular expression by state elimination;
- recognize right-linear and left-linear grammars and convert between regular grammars and finite automata; and
- explain why these formalisms define exactly the regular languages.

Part 1: Inductive Definition of RE

Regular expressions (REs) provide a declarative, algebraic way to describe languages. We define them inductively based on the regular operations.

Part 1: Inductive Definition of RE

Regular expressions (REs) provide a declarative, algebraic way to describe languages. We define them inductively based on the regular operations.

Formal Definition

Given an alphabet Σ , R is a regular expression if R is built by the following rules. Its denoted language is written $L(R)$.

- 1 a for $a \in \Sigma$, with $L(a) = \{a\}$;
- 2 ε , with $L(\varepsilon) = \{\varepsilon\}$;
- 3 $\emptyset$, with $L(\emptyset) = \emptyset$;
- 4 $(R_1 \cup R_2)$, with $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$;
- 5 $(R_1 \circ R_2)$, with $L(R_1 \circ R_2) = L(R_1)L(R_2)$;
- 6 (R_1^*) , with $L(R_1^*) = [L(R_1)]^*$.

Part 1: Inductive Definition of RE

Regular expressions (REs) provide a declarative, algebraic way to describe languages. We define them inductively based on the regular operations.

Formal Definition

Given an alphabet Σ , R is a regular expression if R is built by the following rules. Its denoted language is written $L(R)$.

- 1 a for $a \in \Sigma$, with $L(a) = \{a\}$;
- 2 ε , with $L(\varepsilon) = \{\varepsilon\}$;
- 3 $\emptyset$, with $L(\emptyset) = \emptyset$;
- 4 $(R_1 \cup R_2)$, with $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$;
- 5 $(R_1 \circ R_2)$, with $L(R_1 \circ R_2) = L(R_1)L(R_2)$;
- 6 (R_1^*) , with $L(R_1^*) = [L(R_1)]^*$.

Notation Note

We use $\cup$ for union and ε for the empty string. Some texts instead use $+$ for union and λ for the empty string.

Precedence Rules and Empty-Language Notation

To avoid excessively nested parentheses, we establish an order of operations:

Star (*) > **Concatenation (◦)** > **Union (∪)**

For example, $0 \cup 10^*$ is parsed as $0 \cup (1 \circ (0^*))$, not $(0 \cup 1) \circ 0^*$.

Precedence Rules and Empty-Language Notation

To avoid excessively nested parentheses, we establish an order of operations:

Star (*) > **Concatenation (o)** > **Union (U)**

For example, $0 \cup 10^*$ is parsed as $0 \cup (1 \circ (0^*))$, not $(0 \cup 1) \circ 0^*$.

Teacher's Corner: $\emptyset$ vs. $\{\varepsilon\}$

A frequent source of confusion is the distinction between the expressions $\emptyset$ and ε and the languages they denote.

- $L(\emptyset) = \emptyset$ contains **no strings**.
- $L(\varepsilon) = \{\varepsilon\}$ contains **exactly one string**: the empty string.
- Concatenation identities: $R \circ \varepsilon = R$, but $R \circ \emptyset = \emptyset$.
- Union identities: $R \cup \emptyset = R$, but $R \cup \varepsilon$ adds the empty string to $L(R)$.

Useful Algebraic Laws for Regular Expressions

These are language equivalences: $R \equiv S$ means $L(R) = L(S)$.

Identity and Annihilation

$$\begin{aligned}R \cup \emptyset &\equiv R, & R \cup R &\equiv R, \\ R\epsilon &\equiv \epsilon R \equiv R, \\ R\emptyset &\equiv \emptyset R \equiv \emptyset.\end{aligned}$$

Star and Distribution

$$\begin{aligned}\emptyset^* &\equiv \epsilon, & \epsilon^* &\equiv \epsilon, \\ R(S \cup T) &\equiv RS \cup RT, \\ (R \cup S)T &\equiv RT \cup ST.\end{aligned}$$

Important Non-Law

Concatenation is generally not commutative: $RS \not\equiv SR$. These laws simplify state-elimination labels and help check answers.

Examples of Regular Expressions

Let $\Sigma = \{0, 1\}$. How do we express common patterns algebraically?

Examples of Regular Expressions

Let $\Sigma = \{0, 1\}$. How do we express common patterns algebraically?

1. Strings containing at least one 00 as a substring:

$$(0 \cup 1)^* 00 (0 \cup 1)^*$$

Explanation: The $(0 \cup 1)^*$ acts as a wildcard Σ^* representing any prefix and any suffix.

Examples of Regular Expressions

Let $\Sigma = \{0, 1\}$. How do we express common patterns algebraically?

1. Strings containing at least one 00 as a substring:

$$(0 \cup 1)^* 00 (0 \cup 1)^*$$

Explanation: The $(0 \cup 1)^*$ acts as a wildcard Σ^* representing any prefix and any suffix.

2. Strings that do not contain 00 as a substring:

$$(1 \cup 01)^* (0 \cup \varepsilon)$$

Explanation: Every nonfinal 0 is followed by a 1; at most one unmatched 0 may occur, and only at the end.

Examples of Regular Expressions

Let $\Sigma = \{0, 1\}$. How do we express common patterns algebraically?

1. Strings containing at least one 00 as a substring:

$$(0 \cup 1)^* 00 (0 \cup 1)^*$$

Explanation: The $(0 \cup 1)^*$ acts as a wildcard Σ^* representing any prefix and any suffix.

2. Strings that do not contain 00 as a substring:

$$(1 \cup 01)^* (0 \cup \varepsilon)$$

Explanation: Every nonfinal 0 is followed by a 1; at most one unmatched 0 may occur, and only at the end.

3. Strings with an even number of 0s:

$$1^* (01^* 01^*)^*$$

Explanation: 1s can appear anywhere. 0s must appear in pairs (01^*0) to ensure their total count is even.

Part 2: RE to NFA — Thompson's Construction

Kleene's Theorem: A language is regular if and only if it is described by a regular expression.

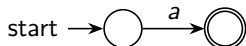
Here we prove the direction *regular expression* $\Rightarrow$ *regular language* by converting any RE into an equivalent ε -NFA. First, we establish the atomic primitives:

Part 2: RE to NFA — Thompson's Construction

Kleene's Theorem: A language is regular if and only if it is described by a regular expression.

Here we prove the direction *regular expression* $\Rightarrow$ *regular language* by converting any RE into an equivalent ε -NFA. First, we establish the atomic primitives:

RE: a for $a \in \Sigma$

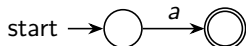


Part 2: RE to NFA — Thompson's Construction

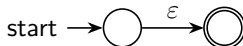
Kleene's Theorem: A language is regular if and only if it is described by a regular expression.

Here we prove the direction *regular expression* $\Rightarrow$ *regular language* by converting any RE into an equivalent ε -NFA. First, we establish the atomic primitives:

RE: a for $a \in \Sigma$



RE: ε

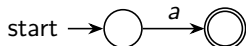


Part 2: RE to NFA — Thompson's Construction

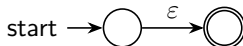
Kleene's Theorem: A language is regular if and only if it is described by a regular expression.

Here we prove the direction *regular expression* $\Rightarrow$ *regular language* by converting any RE into an equivalent ε -NFA. First, we establish the atomic primitives:

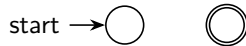
RE: a for $a \in \Sigma$



RE: ε



RE: $\emptyset$



Each primitive has one start state and one distinct accept state; the $\emptyset$ machine simply has no path between them.

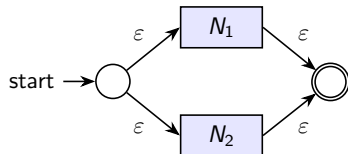
Inductive Steps: Union and Concatenation

Assume we have NFAs N_1 and N_2 for expressions R_1 and R_2 .

Inductive Steps: Union and Concatenation

Assume we have NFAs N_1 and N_2 for expressions R_1 and R_2 .

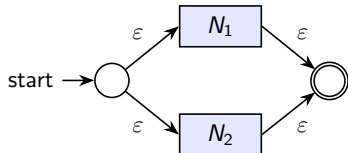
Union ($R_1 \cup R_2$)



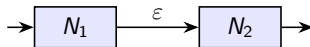
Inductive Steps: Union and Concatenation

Assume we have NFAs N_1 and N_2 for expressions R_1 and R_2 .

Union ($R_1 \cup R_2$)



Concatenation ($R_1 \circ R_2$)



We use ϵ -transitions to seamlessly connect the sub-machines without accidentally reading an input symbol.

Inductive Steps: The Star Operation

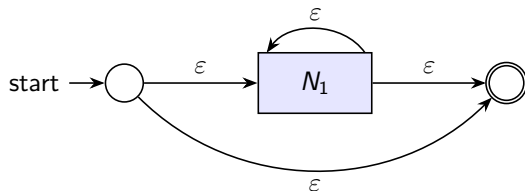
Star Operation (R_1^*)

The Star operation requires us to accept the empty string ε (zero repetitions), or loop back to the beginning of N_1 after a successful pass (one or more repetitions).

Inductive Steps: The Star Operation

Star Operation (R_1^*)

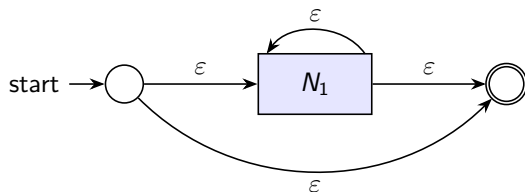
The Star operation requires us to accept the empty string ϵ (zero repetitions), or loop back to the beginning of N_1 after a successful pass (one or more repetitions).



Inductive Steps: The Star Operation

Star Operation (R_1^*)

The Star operation requires us to accept the empty string ϵ (zero repetitions), or loop back to the beginning of N_1 after a successful pass (one or more repetitions).



Why new boundary states? The new start has no incoming transitions and the new accept has no outgoing transitions. This preserves the one-entry/one-exit form needed by later inductive steps.

Worked Example: Converting $(ab \cup a)^*$

Let's build an NFA for $(ab \cup a)^*$ stage-by-stage.

Worked Example: Converting $(ab \cup a)^*$

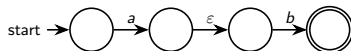
Let's build an NFA for $(ab \cup a)^*$ stage-by-stage.

1. Build the primitive machines for a and b .

Worked Example: Converting $(ab \cup a)^*$

Let's build an NFA for $(ab \cup a)^*$ stage-by-stage.

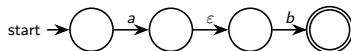
1. **Build the primitive machines for a and b .**
2. **Concatenate them to obtain ab :**



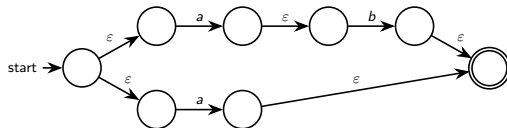
Worked Example: Converting $(ab \cup a)^*$

Let's build an NFA for $(ab \cup a)^*$ stage-by-stage.

1. Build the primitive machines for a and b .
2. Concatenate them to obtain ab :



3. Form $U = (ab \cup a)$:

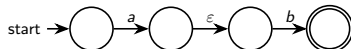


Worked Example: Converting $(ab \cup a)^*$

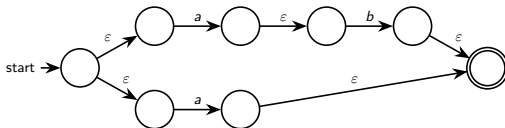
Let's build an NFA for $(ab \cup a)^*$ stage-by-stage.

1. Build the primitive machines for a and b .

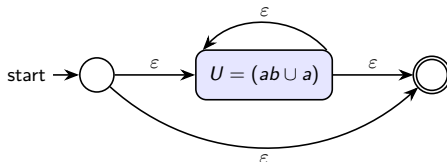
2. Concatenate them to obtain ab :



3. Form $U = (ab \cup a)$:



4. Apply the star wrapper to U :



Part 3: Generalized NFAs (GNFA)

To convert a finite automaton back into a regular expression, we use a **generalized NFA (GNFA)**.

Part 3: Generalized NFAs (GNFA)

To convert a finite automaton back into a regular expression, we use a **generalized NFA (GNFA)**.

What is a GNFA?

A GNFA has transition arrows labeled by **regular expressions** rather than single symbols. Traversing an edge consumes any string in the language denoted by its label.

Part 3: Generalized NFAs (GNFA)

To convert a finite automaton back into a regular expression, we use a **generalized NFA (GNFA)**.

What is a GNFA?

A GNFA has transition arrows labeled by **regular expressions** rather than single symbols. Traversing an edge consumes any string in the language denoted by its label.

Standard Form for GNFA conversion:

- 1 A new **start state** with an ϵ -arrow to the old start state, and no incoming arrows.
- 2 A single new **accept state** with ϵ -arrows from the old accept states, and no outgoing arrows.
- 3 Combine parallel arrows by union and regard a missing arrow as one labeled $\emptyset$. Thus each allowable ordered pair has one RE label; the start has no incoming arrows and the accept state has no outgoing arrows.

The Conversion Algorithm: State Elimination

Objective: Reduce the machine to two states (start and accept) by eliminating internal states one at a time.

The Conversion Algorithm: State Elimination

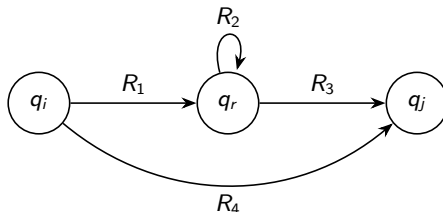
Objective: Reduce the machine to two states (start and accept) by eliminating internal states one at a time.

When we eliminate a state q_r , we update the graph so that its language remains unchanged. To compute the new label on $q_i \rightarrow q_j$:

The Conversion Algorithm: State Elimination

Objective: Reduce the machine to two states (start and accept) by eliminating internal states one at a time.

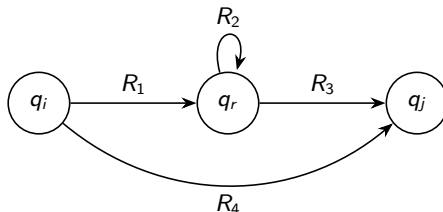
When we eliminate a state q_r , we update the graph so that its language remains unchanged. To compute the new label on $q_i \rightarrow q_j$:



The Conversion Algorithm: State Elimination

Objective: Reduce the machine to two states (start and accept) by eliminating internal states one at a time.

When we eliminate a state q_r , we update the graph so that its language remains unchanged. To compute the new label on $q_i \rightarrow q_j$:



The Update Formula

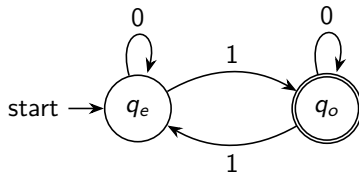
The new label bypassing q_r from q_i to q_j becomes:

$$R' = R_4 \cup R_1 R_2^* R_3$$

Compute this update for **every pair** (q_i, q_j) of remaining states. Different elimination orders may produce different-looking but equivalent expressions.

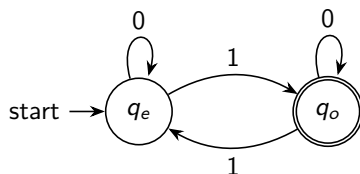
Worked Example: DFA to GNFA

Given DFA: Accepts strings with an odd number of 1s.

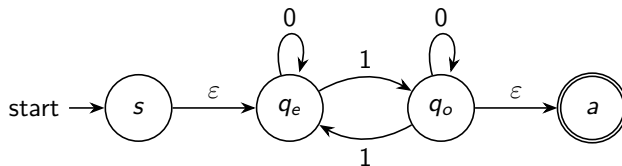


Worked Example: DFA to GNFA

Given DFA: Accepts strings with an odd number of 1s.



Step 1: Convert to Standard GNFA (Add new start s and accept a).



Worked Example: State Elimination

Step 2: Eliminate q_o .

Update the labels on $q_e \rightarrow q_e$ and $q_e \rightarrow a$.

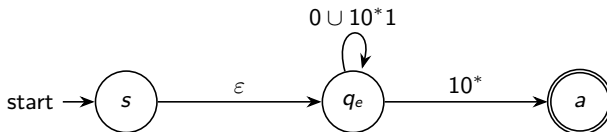
- $q_e \rightarrow q_e$: the bypass through q_o contributes 10^*1 . Union with the existing 0 to obtain $0 \cup 10^*1$.
- $q_e \rightarrow a$: the bypass contributes $10^*\varepsilon = 10^*$. Union with $\emptyset$ to obtain 10^* .

Worked Example: State Elimination

Step 2: Eliminate q_o .

Update the labels on $q_e \rightarrow q_e$ and $q_e \rightarrow a$.

- $q_e \rightarrow q_e$: the bypass through q_o contributes 10^*1 . Union with the existing 0 to obtain $0 \cup 10^*1$.
- $q_e \rightarrow a$: the bypass contributes $10^*\varepsilon = 10^*$. Union with $\emptyset$ to obtain 10^* .

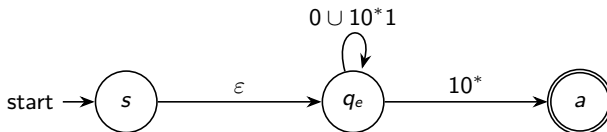


Worked Example: State Elimination

Step 2: Eliminate q_o .

Update the labels on $q_e \rightarrow q_e$ and $q_e \rightarrow a$.

- $q_e \rightarrow q_e$: the bypass through q_o contributes 10^*1 . Union with the existing 0 to obtain $0 \cup 10^*1$.
- $q_e \rightarrow a$: the bypass contributes $10^*\varepsilon = 10^*$. Union with $\emptyset$ to obtain 10^* .



Step 3: Eliminate q_e .

Now we update the edge $s \rightarrow a$.

Path: $s \xrightarrow{\varepsilon} q_e \xrightarrow{(0 \cup 10^*1)^*} q_e \xrightarrow{10^*} a$.

Final RE: $(0 \cup 10^*1)^*10^*$

Part 4: Definitions of Regular Grammars

We have seen that DFAs, NFAs, and REs all describe regular languages. We can also define this class syntactically using restricted grammars.

Part 4: Definitions of Regular Grammars

We have seen that DFAs, NFAs, and REs all describe regular languages. We can also define this class syntactically using restricted grammars.

Regular Grammar Definition

A grammar $G = (V, T, S, P)$ is **regular** if all of its productions are right-linear or all of its productions are left-linear.

Right-Linear: All productions are of the form:

- $A \rightarrow xB$ ($A, B \in V, x \in T^*$)
- $A \rightarrow x$

Left-Linear: All productions are of the form:

- $A \rightarrow Bx$
- $A \rightarrow x$

Part 4: Definitions of Regular Grammars

We have seen that DFAs, NFAs, and REs all describe regular languages. We can also define this class syntactically using restricted grammars.

Regular Grammar Definition

A grammar $G = (V, T, S, P)$ is **regular** if all of its productions are right-linear or all of its productions are left-linear.

Right-Linear: All productions are of the form:

- $A \rightarrow xB$ ($A, B \in V, x \in T^*$)
- $A \rightarrow x$

Left-Linear: All productions are of the form:

- $A \rightarrow Bx$
- $A \rightarrow x$

Important: At most one variable appears on each right-hand side, and its orientation must be consistent throughout the grammar. Having at most one variable is necessary, but not sufficient, for a grammar to be regular.

Right-Linear, Left-Linear, or Merely Linear?

Right-linear example

$$S \rightarrow abS \mid a$$

Every variable is at the right end. The generated language is

$$L = (ab)^*a.$$

Left-linear example

$$S \rightarrow Aab,$$

$$A \rightarrow Aab \mid B,$$

$$B \rightarrow a.$$

Every variable is at the left end. Here $L = aab(ab)^*$.

Right-Linear, Left-Linear, or Merely Linear?

Right-linear example

$$S \rightarrow abS \mid a$$

Every variable is at the right end. The generated language is

$$L = (ab)^*a.$$

Left-linear example

$$S \rightarrow Aab,$$

$$A \rightarrow Aab \mid B,$$

$$B \rightarrow a.$$

Every variable is at the left end. Here $L = aab(ab)^*$.

Do Not Mix the Two Orientations

The grammar with rules $S \rightarrow aA$, $A \rightarrow Sb$, and $S \rightarrow \epsilon$ is linear but not a regular grammar: it mixes right- and left-linear productions. In fact, it generates the nonregular language $\{a^n b^n : n \geq 0\}$.

Reading Derivations in a Regular Grammar

Consider the right-linear grammar

$$S \rightarrow abS \mid a.$$

Reading Derivations in a Regular Grammar

Consider the right-linear grammar

$$S \rightarrow abS \mid a.$$

Each use of $S \rightarrow abS$ contributes one block ab and keeps the derivation active. The rule $S \rightarrow a$ terminates it.

Sample Derivations

$$S \Rightarrow a,$$

$$S \Rightarrow abS \Rightarrow aba,$$

$$S \Rightarrow abS \Rightarrow ababS \Rightarrow ababab.$$

Reading Derivations in a Regular Grammar

Consider the right-linear grammar

$$S \rightarrow abS \mid a.$$

Each use of $S \rightarrow abS$ contributes one block ab and keeps the derivation active. The rule $S \rightarrow a$ terminates it.

Sample Derivations

$$S \Rightarrow a,$$

$$S \Rightarrow abS \Rightarrow aba,$$

$$S \Rightarrow abS \Rightarrow ababS \Rightarrow ababab.$$

Thus every completed derivation has the form $(ab)^n a$, and every $(ab)^n a$ can be derived. Therefore $L(G) = (ab)^* a$.

Theorem: Right-Linear Grammars Generate Regular Languages

Theorem

If $G = (V, T, S, P)$ is right-linear, then $L(G)$ is a regular language.

Theorem: Right-Linear Grammars Generate Regular Languages

Theorem

If $G = (V, T, S, P)$ is right-linear, then $L(G)$ is a regular language.

Proof outline: first normalize terminal blocks, then construct an equivalent NFA.

- 1 Use one state for each variable and make S the start state.
- 2 For $A \rightarrow aB$, add the transition $A \xrightarrow{a} B$.
- 3 For $A \rightarrow B$, add $A \xrightarrow{\varepsilon} B$.
- 4 For $A \rightarrow a$, add $A \xrightarrow{a} F$ to one new accept state F .
- 5 For $A \rightarrow \varepsilon$, either make A accepting or add $A \xrightarrow{\varepsilon} F$.

Theorem: Right-Linear Grammars Generate Regular Languages

Theorem

If $G = (V, T, S, P)$ is right-linear, then $L(G)$ is a regular language.

Proof outline: first normalize terminal blocks, then construct an equivalent NFA.

- 1 Use one state for each variable and make S the start state.
- 2 For $A \rightarrow aB$, add the transition $A \xrightarrow{a} B$.
- 3 For $A \rightarrow B$, add $A \xrightarrow{\varepsilon} B$.
- 4 For $A \rightarrow a$, add $A \xrightarrow{a} F$ to one new accept state F .
- 5 For $A \rightarrow \varepsilon$, either make A accepting or add $A \xrightarrow{\varepsilon} F$.

Why the Construction Works

After reading w , the NFA can be in state A exactly when $S \Rightarrow^* wA$. Reaching an accept state corresponds precisely to completing a derivation $S \Rightarrow^* w$.

Normalizing Long Terminal Blocks

Some definitions permit productions $A \rightarrow xB$ and $A \rightarrow x$ with $x \in T^*$. Before drawing an NFA, split each long terminal block into one-symbol steps.

Normalizing Long Terminal Blocks

Some definitions permit productions $A \rightarrow xB$ and $A \rightarrow x$ with $x \in T^*$. Before drawing an NFA, split each long terminal block into one-symbol steps.

General Replacement

For $k \geq 2$, replace

$$A \rightarrow a_1 a_2 \cdots a_k B$$

by fresh variables $C_1, \dots, C_{k-1}$ and rules

$$A \rightarrow a_1 C_1, \quad C_1 \rightarrow a_2 C_2, \quad \dots, \quad C_{k-1} \rightarrow a_k B.$$

For a terminal-only rule $A \rightarrow a_1 \cdots a_k$, use the same chain but end with $C_{k-1} \rightarrow a_k$ (or a transition to the new final state in the NFA construction).

Normalizing Long Terminal Blocks

Some definitions permit productions $A \rightarrow xB$ and $A \rightarrow x$ with $x \in T^*$. Before drawing an NFA, split each long terminal block into one-symbol steps.

General Replacement

For $k \geq 2$, replace

$$A \rightarrow a_1 a_2 \cdots a_k B$$

by fresh variables $C_1, \dots, C_{k-1}$ and rules

$$A \rightarrow a_1 C_1, \quad C_1 \rightarrow a_2 C_2, \quad \dots, \quad C_{k-1} \rightarrow a_k B.$$

For a terminal-only rule $A \rightarrow a_1 \cdots a_k$, use the same chain but end with $C_{k-1} \rightarrow a_k$ (or a transition to the new final state in the NFA construction).

Example

$V_1 \rightarrow abV_0$ becomes $V_1 \rightarrow aV_2$ and $V_2 \rightarrow bV_0$. The introduced variable records progress through the terminal block.

Worked Example: Regular Grammar to NFA

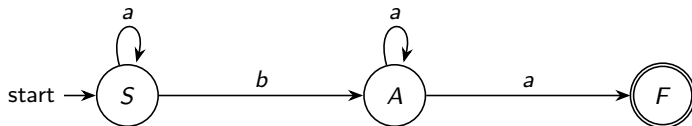
Consider

$$S \rightarrow aS \mid bA, \quad A \rightarrow aA \mid a.$$

Worked Example: Regular Grammar to NFA

Consider

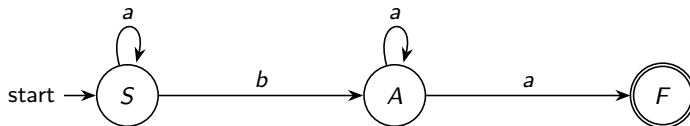
$$S \rightarrow aS \mid bA, \quad A \rightarrow aA \mid a.$$



Worked Example: Regular Grammar to NFA

Consider

$$S \rightarrow aS \mid bA, \quad A \rightarrow aA \mid a.$$



Derivation

$$\begin{aligned} S &\Rightarrow aS \Rightarrow aaS \Rightarrow aabA \\ &\Rightarrow aabaA \Rightarrow aabaa. \end{aligned}$$

Language

$$L(G) = a^*ba^+ = a^*baa^*.$$

Theorem: Every Regular Language Has a Right-Linear Grammar

Theorem

If $L \subseteq \Sigma^*$ is regular, then some right-linear grammar $G = (V, \Sigma, S, P)$ satisfies $L = L(G)$.

Theorem: Every Regular Language Has a Right-Linear Grammar

Theorem

If $L \subseteq \Sigma^*$ is regular, then some right-linear grammar $G = (V, \Sigma, S, P)$ satisfies $L = L(G)$.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA for L .

- 1 Create one variable Q_i for every automaton state q_i .
- 2 Use Q_0 as the grammar's start variable.
- 3 For each transition $q_i \xrightarrow{a} q_j$, add $Q_i \rightarrow aQ_j$.
- 4 For each ε -transition $q_i \xrightarrow{\varepsilon} q_j$, add $Q_i \rightarrow Q_j$.
- 5 For every accepting state q_f , add $Q_f \rightarrow \varepsilon$.

Theorem: Every Regular Language Has a Right-Linear Grammar

Theorem

If $L \subseteq \Sigma^*$ is regular, then some right-linear grammar $G = (V, \Sigma, S, P)$ satisfies $L = L(G)$.

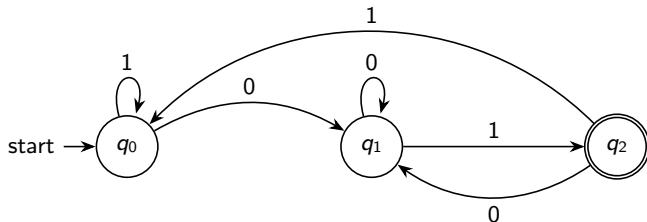
Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA for L .

- 1 Create one variable Q_i for every automaton state q_i .
- 2 Use Q_0 as the grammar's start variable.
- 3 For each transition $q_i \xrightarrow{a} q_j$, add $Q_i \rightarrow aQ_j$.
- 4 For each ε -transition $q_i \xrightarrow{\varepsilon} q_j$, add $Q_i \rightarrow Q_j$.
- 5 For every accepting state q_f , add $Q_f \rightarrow \varepsilon$.

A path labeled w from q_0 to q_f corresponds step-for-step to a derivation $Q_0 \Rightarrow^* wQ_f \Rightarrow w$.

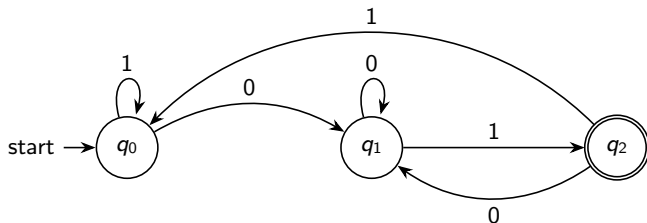
Worked Example: DFA to Right-Linear Grammar

The DFA below accepts exactly the binary strings ending in 01.



Worked Example: DFA to Right-Linear Grammar

The DFA below accepts exactly the binary strings ending in 01.



Replace states by variables Q_0, Q_1, Q_2 :

$$Q_0 \rightarrow 0Q_1 \mid 1Q_0,$$

$$Q_1 \rightarrow 0Q_1 \mid 1Q_2,$$

$$Q_2 \rightarrow 0Q_1 \mid 1Q_0 \mid \varepsilon.$$

Left-Linear Grammars and the Equivalence Theorem

Left-Linear Version

A language is regular if and only if it is generated by a left-linear grammar.

Left-Linear Grammars and the Equivalence Theorem

Left-Linear Version

A language is regular if and only if it is generated by a left-linear grammar.

For a finite automaton, build productions in the reverse direction:

- If $q_i \xrightarrow{a} q_j$, add $Q_j \rightarrow Q_i a$.
- If $q_i \xrightarrow{\varepsilon} q_j$, add $Q_j \rightarrow Q_i$.
- Introduce a new grammar start S with $S \rightarrow Q_f$ for each accepting state q_f .
- Add $Q_0 \rightarrow \varepsilon$ for the automaton's start state.

Left-Linear Grammars and the Equivalence Theorem

Left-Linear Version

A language is regular if and only if it is generated by a left-linear grammar.

For a finite automaton, build productions in the reverse direction:

- If $q_i \xrightarrow{a} q_j$, add $Q_j \rightarrow Q_i a$.
- If $q_i \xrightarrow{\varepsilon} q_j$, add $Q_j \rightarrow Q_i$.
- Introduce a new grammar start S with $S \rightarrow Q_f$ for each accepting state q_f .
- Add $Q_0 \rightarrow \varepsilon$ for the automaton's start state.

Combined Equivalence

For every language L , the following are equivalent:

$$\begin{aligned} L \text{ is regular} &\iff L = L(G_R) \text{ for some right-linear } G_R \\ &\iff L = L(G_L) \text{ for some left-linear } G_L. \end{aligned}$$

Regular-Grammar Checkpoint Exercises

- 1 Classify each grammar as right-linear, left-linear, linear but not regular, or not linear:
 - 1 $S \rightarrow 0S \mid 1A, A \rightarrow 1;$
 - 2 $S \rightarrow A0, A \rightarrow A1 \mid \varepsilon;$
 - 3 $S \rightarrow 0A, A \rightarrow S1 \mid \varepsilon.$
- 2 Convert $S \rightarrow 0S \mid 1A, A \rightarrow 0A \mid 1$ into an NFA and describe its language.
- 3 Construct a right-linear grammar for all binary strings that end in 00.

Regular-Grammar Checkpoint Exercises

- ① Classify each grammar as right-linear, left-linear, linear but not regular, or not linear:
 - ① $S \rightarrow 0S \mid 1A, A \rightarrow 1;$
 - ② $S \rightarrow A0, A \rightarrow A1 \mid \varepsilon;$
 - ③ $S \rightarrow 0A, A \rightarrow S1 \mid \varepsilon.$
- ② Convert $S \rightarrow 0S \mid 1A, A \rightarrow 0A \mid 1$ into an NFA and describe its language.
- ③ Construct a right-linear grammar for all binary strings that end in 00.

Proof Practice

State the invariant that connects a partial derivation $S \Rightarrow^* wA$ with a path in the corresponding NFA.

Hints: Regular-Grammar Checkpoint

Hints

- Exercise 1(c) mixes orientations; try deriving several strings to see why “one variable per right side” is not sufficient.
- For Exercise 2, use states S , A , and a new final state F . The resulting regular expression is 0^*10^*1 .
- For Exercise 3, first draw the three-state suffix-tracking DFA. Then convert every transition $Q_i \xrightarrow{a} Q_j$ into $Q_i \rightarrow aQ_j$ and add ε at the accept state.

Hints: Regular-Grammar Checkpoint

Hints

- Exercise 1(c) mixes orientations; try deriving several strings to see why “one variable per right side” is not sufficient.
- For Exercise 2, use states S , A , and a new final state F . The resulting regular expression is 0^*10^*1 .
- For Exercise 3, first draw the three-state suffix-tracking DFA. Then convert every transition $Q_i \xrightarrow{a} Q_j$ into $Q_i \rightarrow aQ_j$ and add ε at the accept state.

Invariant for the Proof

For every $w \in T^*$ and $A \in V$,

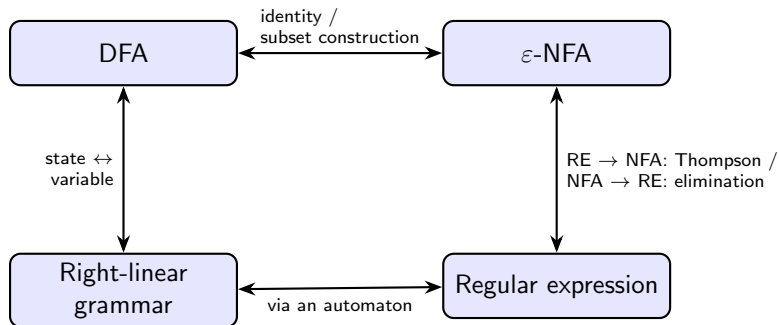
$$S \Rightarrow^* wA \iff \text{the NFA has a path } S \xrightarrow{w} A.$$

Part 5: The Equivalence Map

Four formalisms describe exactly the regular languages.

Part 5: The Equivalence Map

Four formalisms describe exactly the regular languages.



Applications of Regular Languages

Why do we care about regular expressions and finite automata?

Why do we care about regular expressions and finite automata?

1. Compiler Design (Lexical Analysis)

Compilers use regular expressions to define the tokens of a programming language (e.g., identifiers, constants, and reserved words).

If $D = (0 \cup \dots \cup 9)$, a simple signed decimal can be described by $(\epsilon \cup + \cup -)DD^*(\epsilon \cup .DD^*)$. Tools such as `lex` convert token specifications into efficient finite-automaton-based scanners.

Applications of Regular Languages

Why do we care about regular expressions and finite automata?

1. Compiler Design (Lexical Analysis)

Compilers use regular expressions to define the tokens of a programming language (e.g., identifiers, constants, and reserved words).

If $D = (0 \cup \dots \cup 9)$, a simple signed decimal can be described by $(\epsilon \cup + \cup -)DD^*(\epsilon \cup .DD^*)$. Tools such as `lex` convert token specifications into efficient finite-automaton-based scanners.

2. Pattern Matching and Text Editors

Classical regular expressions can be matched in linear time in the text length using automaton-based methods. Many programming-language “regex” engines also support backreferences, lookahead, and backtracking; those extensions may exceed regular-language power and need not run in linear time.

Checkpoint Exercises

- 1 **GNFA Conversion:** Given a DFA that accepts strings ending in 1, with states q_0 (start) and q_1 (accept), the transitions are $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_0$, and $\delta(q_1, 1) = q_1$. Convert this DFA to a regular expression using state elimination.

- 2 **Grammars:** Write a right-linear grammar for the language:

$$L = \{w \in \{0, 1\}^* \mid w \text{ ends in } 01\}.$$

Hints

- **Exercise 1:**

- Add the standard GNFA start (s) and accept (a) states.
- When ripping q_1 , the path $q_0 \rightarrow q_1 \rightarrow q_0$ creates a loop on q_0 labeled 11^*0 .
- The total loop on q_0 becomes $0 \cup 11^*0$.
- The path to a is via $q_1 \rightarrow a$ which is ε , so $q_0 \rightarrow q_1 \rightarrow a$ gives 11^* .
- Eliminating q_0 next gives $(0 \cup 11^*0)^*11^*$, which is equivalent to $(0 \cup 1)^*1$.

- **Exercise 2:** One answer is $S \rightarrow 0S \mid 1S \mid 0A$, $A \rightarrow 1B$, and $B \rightarrow \varepsilon$. The nondeterministic choice on 0 guesses the start of the final suffix 01.

Hints for Checkpoint Exercises

Hints

• Exercise 1:

- Add the standard GNFA start (s) and accept (a) states.
- When ripping q_1 , the path $q_0 \rightarrow q_1 \rightarrow q_0$ creates a loop on q_0 labeled 11^*0 .
- The total loop on q_0 becomes $0 \cup 11^*0$.
- The path to a is via $q_1 \rightarrow a$ which is ε , so $q_0 \rightarrow q_1 \rightarrow a$ gives 11^* .
- Eliminating q_0 next gives $(0 \cup 11^*0)^*11^*$, which is equivalent to $(0 \cup 1)^*1$.

- **Exercise 2:** One answer is $S \rightarrow 0S \mid 1S \mid 0A$, $A \rightarrow 1B$, and $B \rightarrow \varepsilon$. The nondeterministic choice on 0 guesses the start of the final suffix 01.

Final Pro-Tip for State Elimination

Simplify regular-expression labels after each state-elimination update. For instance, replace εR by R and $R \cup \emptyset$ by R . This keeps the final expression readable and easier to verify.